Nifty 100 Financial Intelligence Platform

Analyst Guide

Version 1.0 — Sprint 6

1. Overview

The Nifty 100 Financial Intelligence Platform is a self-contained analytics system covering 92 Nifty 100 companies across 12 years of financial history. It combines a validated SQLite database, a computed ratio engine, an investment screener, peer comparison tools, automated PDF reporting, a Streamlit dashboard, and a REST API.

What this guide covers

- Using the Streamlit screener and dashboard screens
- Navigating each of the 8 dashboard screens
- Generating PDF tearsheets and reports
- Calling the REST API with example commands
- Troubleshooting common issues

2. Getting Started

All commands below assume you're in the project's root folder with the virtual environment activated.

First-time setup

```
python -m venv venv
venv\Scripts\pip install -r requirements.txt
copy .env.example .env
```

Build the database

```
venv\Scripts\python -m src.etl.loader
venv\Scripts\python -m src.analytics.populate_ratios
```

This must be run before anything else — every downstream module (screener, dashboard, API) reads from the resulting nifty100.db file.

3. Using the Streamlit Screener

Launch the dashboard with:

```
venv\Scripts\streamlit run src\dashboard\app.py
```

Navigate to the Screener page from the sidebar. Ten sliders let you set thresholds for ROE, debt-to-equity, free cash flow, revenue and profit growth, margins, valuation multiples, dividend yield, and interest coverage.

Using presets

Six preset buttons — Quality, Value, Growth, Dividend, Debt-Free, and Turnaround — auto-fill the sliders to match a known investment style. You can still fine-tune any slider afterward.

Exporting results

The Download CSV button exports exactly what's shown in the results table, including the composite quality score column.

4. Navigating the Dashboard

The dashboard has 8 screens, accessible from the sidebar:

- **Home** — portfolio-wide KPI tiles, sector donut chart, top 5 by composite score, year selector
- **Profile** — search any company for its KPIs, 10-year charts, and pros/cons
- **Screener** — see Section 3
- **Peers** — radar chart of a company vs its peer group average, side-by-side table
- **Trends** — overlay up to 3 metrics for one company with YoY % change labels
- **Sectors** — bubble chart (Revenue x ROE, sized by market cap) plus median KPI bars
- **Capital** — treemap of all companies by capital allocation pattern
- **Reports** — annual report links per company, with an on-demand dead-link check

5. Generating PDF Reports

Three types of PDF reports can be generated from the command line:

Company tearsheets (2 pages, one per company)

```
venv\Scripts\python -m src.reports.tearsheet
```

Companies with fewer than 3 years of P&L; history are skipped and logged to output/skipped_tearsheets.csv — this is expected, not an error.

Sector reports (one per broad sector)

```
venv\Scripts\python -m src.reports.sector_report
```

Portfolio summary (one page per company)

```
venv\Scripts\python -m src.reports.portfolio_summary
```

Each page shows a company's top 6 KPIs with a colour-coded UP/DOWN/FLAT trend label versus the prior year.

6. Using the REST API

Start the API server with:

```
venv\Scripts\uvicorn src.api.main:app --port 8000
```

Interactive documentation is available at http://localhost:8000/docs — every endpoint can be tried directly from the browser.

Example: get a company's ratio history

```
curl "http://localhost:8000/api/v1/companies/TCS/ratios"
```

Example: run the screener via the API

```
curl "http://localhost:8000/api/v1/screener?min_roe=15&max;_de=1"
```

Example: download a tearsheet

```
curl "http://localhost:8000/api/v1/companies/TCS/tearsheet" --output tcs.pdf
```

All 16 endpoints are documented in docs/openapi.json and can be imported into Postman via docs/postman_collection.json.

7. Understanding the Data

A few things worth knowing before drawing conclusions from the numbers:

- The dataset covers 92 of the 100 Nifty 100 companies — 8 were excluded during data collection for availability reasons.
- Rows labelled 'TTM' (Trailing Twelve Months) in the source data are intentionally excluded from annual tables, since they aren't a complete fiscal year.
- The sector taxonomy has 10 broad sectors in the actual dataset, not the 11 originally anticipated — the 'Conglomerates/Other' category isn't present in the source file.
- market_cap.xlsx and stock_prices.xlsx are simulated data, clearly labelled as such — don't draw real investment conclusions from valuation multiples or price charts.
- CON-11 in the auto-generated pros/cons ('Net Debt > 3x EBITDA') uses operating_profit as an EBITDA proxy, per this project's own glossary definition.

8. Troubleshooting

"Database not found" errors

Run `make load` (or the manual loader command) first — every other module depends on nifty100.db existing.

Dashboard shows 'N/A' for a company

This is expected for companies with partial data coverage — it means that specific metric couldn't be computed (e.g. division by zero, or a missing source value), not a bug.

Port already in use

The dashboard defaults to port 8501, the API to port 8000. If either is already running elsewhere on your machine, stop that process or specify a different port (--server.port for Streamlit, --port for uvicorn).

Tests failing after a fresh clone

Make sure you've run `pip install -r requirements.txt` inside your virtual environment, and that you're running pytest from the project root, not a subfolder.

Getting help

Check output/load_audit.csv and output/validation_failures.csv first — most data-related questions are answered by what's already logged there.

9. Project Architecture Summary

For reference, here's how the six build sprints fit together:

- **Sprint 1 — Data Foundation:** loads 12 source Excel files into a validated 12-table SQLite database, enforcing 16 data quality rules.
- **Sprint 2 — Ratio Engine:** computes 50+ financial ratios per company-year, including CAGR growth metrics and capital allocation classification.
- **Sprint 3 — Screener & Peers:** 6 preset screeners plus custom filtering, and percentile ranking within 11 peer groups.
- **Sprint 4 — Dashboard & Valuation:** an 8-screen Streamlit app and a valuation module flagging over/undervalued companies vs sector medians.
- **Sprint 5 — NLP & Reports:** auto-generated pros/cons (24 rules), cash flow intelligence scoring, and automated PDF tearsheets, sector reports, and portfolio summaries.
- **Sprint 6 — Clustering, API & QA:** KMeans company archetypes, a 16-endpoint REST API, and this documentation.

Where things live

```
src/etl/ - loading, normalising, validating source data
src/analytics/ - ratio engine, screener scoring, clustering, valuation
src/nlp/ - text parsing and rule-based insight generation
src/reports/ - PDF generation (tearsheets, sector, portfolio, this guide)
src/dashboard/ - Streamlit app and pages
src/api/ - FastAPI server and routers
tests/ - one folder per module area, 170+ tests total
```